SITUAÇÃO DE APRENDIZAGEM	
Segmento:	Técnico
Unidade (s) Curricular (es):	Desenvolvimento de Sistemas
Curso (s):	Desenvolvimento de Sistemas
Instrutor (a):	Vitor Hugo Almeida de Sousa

ESTRATÉGIA DE APRENDIZAGEM DESAFIADORA

[x] Situação-problema [ ] Estudo do Caso [ ] Projeto [ ] Pesquisa Aplicada

DESCRIÇÃO DA ESTRATÉGIA DE APRENDIZAGEM DESAFIADORA

Contextualização: Validação dos Dados de Setores e Equipamentos

A equipe de suporte já utiliza sua API para cadastrar setores e equipamentos. Porém, ainda é possível enviar nomes vazios, patrimônios em branco ou equipamentos sem setor, comprometendo a qualidade dos registros. Nesta continuação, você aplicará as validações já estudadas ao mesmo projeto. Trabalhe primeiro com Setor e depois com Equipamento, teste no Postman e registre cada avanço no Git.

DESAFIO

Etapa 1 — Conferir o projeto atual

Abra o projeto aberturaordensservico e execute a API com o banco ordens_servico. Confira se os CRUDs de Setor e Equipamento funcionam. Utilize a mesma collection “Abertura de Ordens de Serviço” e mantenha os dados já cadastrados.

Confira no pom.xml a dependência spring-boot-starter-validation, prevista na atividade anterior. Se ela não estiver presente, adicione-a e atualize as dependências Maven. Nesta etapa, mantenha as classes e as rotas existentes.

Etapa 2 — Preparar o Git para a continuação

No Git Bash, dentro da pasta do projeto, execute git status. Se houver alterações da atividade anterior, revise-as e registre um commit antes de continuar. Confira a branch atual com git branch.

Com o CRUD básico concluído e salvo, execute:

- git switch -c validacao-dados
- git push -u origin validacao-dados

Essa branch receberá as validações. A main manterá a versão anterior até a conclusão e conferência dos testes.

Etapa 3 — Definir as regras de validação

Setor: nome obrigatório; não pode ser vazio nem conter apenas espaços.
Equipamento: nome e numeroPatrimonio obrigatórios; não podem ser vazios nem conter apenas espaços. O atributo setor não pode ser nulo.
Use @NotBlank para os textos e @NotNull para setor. @Valid ativa a verificação dessas regras na entrada do Controller.

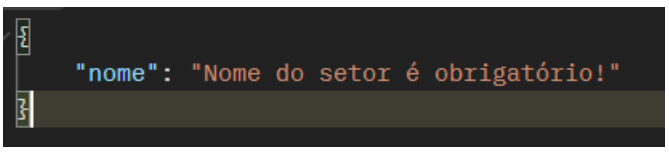
Etapa 4 — Validar Setor e testar

1. Model: em Setor.java, aplique @NotBlank ao atributo nome. Escreva uma mensagem clara no parâmetro message, indicando que o nome do setor é obrigatório. Utilize o import jakarta.validation.constraints.NotBlank.
2. Controller: em SetorController, adicione @Valid ao parâmetro anotado com @RequestBody nos métodos de POST e PUT. Utilize jakarta.validation.Valid.
   Repository e Service: mantenha as operações já implementadas. As regras dos atributos ficam no Model; a validação da entrada é acionada no Controller.
3. Postman: teste POST http://localhost:8080/setores com {"nome":"Costura"}. Depois, envie separadamente {"nome":""}, {"nome":" "}, {"nome":null} e {}. Para os dados inválidos, confira o retorno 400 Bad Request e faça GET da lista para confirmar que não houve cadastro.
4. Teste PUT /setores/{id} com um ID existente: envie primeiro um nome válido e depois repita os casos inválidos do POST. Após cada tentativa rejeitada, consulte o mesmo ID e confirme que o nome anterior foi preservado. Salve os testes na pasta Setores da collection.

Etapa 5 — Personalizar as mensagens de validação

Use como referência as páginas 10 a 12 do material DEV_SIS_10 (Spring Boot - Validação). No pacote principal br.com.ordensservico, crie o pacote exception e a classe ValidacaoExceptionHandler. Adapte o exemplo do material ao pacote deste projeto.

Resultado esperado:



Etapa 6 — Validar Equipamento e testar

1. Model: em Equipamento.java, aplique @NotBlank a nome e numeroPatrimonio e @NotNull a setor. Defina mensagens claras para cada campo. Utilize os imports de jakarta.validation.constraints. Controller: adicione @Valid ao parâmetro @RequestBody nos métodos de POST e PUT de EquipamentoController.
2. Service: mantenha a consulta que verifica se o setor informado existe antes de salvar. @NotNull verifica se o objeto setor foi enviado; não verifica sua existência no banco. Nesta etapa, não coloque @Valid no atributo setor, pois a requisição envia apenas seu ID, sem o nome do setor.
3. No Postman, envie POST /equipamentos com {"nome":"Máquina de Costura","numeroPatrimonio":"PAT-001","setor":{"id":1}}. Substitua 1 por um ID real. Confirme o cadastro e o vínculo com o setor pelo GET.

Altere um campo por vez: teste nome e numeroPatrimonio com "", " ", null e com o campo omitido. Mantenha os demais dados válidos. Depois, teste setor:null e a ausência de setor. Espere 400 Bad Request e confira no GET que nenhum equipamento inválido foi cadastrado.

Resultados esperados:

```
{
    "nome": "Nome do equipamento é obrigatório!",
    "setor": "Setor do equipamento é obrigatório!",
    "numeroPatrimonio": "Número do patrimônio do equipamento é obrigatório!"
}
```

Repita os testes inválidos em PUT /equipamentos/{id}, usando um equipamento existente e enviando os demais campos corretamente. Após cada rejeição, faça GET por ID para confirmar que os dados anteriores foram preservados. Faça também uma atualização válida e confira o resultado.
Envie também nome e numeroPatrimonio vazios e setor:null na mesma requisição. Confira se o JSON de resposta identifica os três campos com as mensagens definidas no Model. O mesmo Handler deve atender Setor e Equipamento.

Teste um setor com ID inexistente, mantendo nome e patrimônio válidos. Confirme que a verificação anterior do Service continua impedindo a gravação. Salve os testes na pasta Equipamentos.

Registre a etapa:

- **git add .**
- **git commit -m "feat: validar dados de setor e equipamento"**
- **git push**

Etapa 7 — Conferir e concluir no Git

Confira novamente GET e DELETE dos dois CRUDs. Na descrição dos testes inválidos, registre o campo testado, o resultado esperado e o observado. Execute git status e git log --oneline para conferir a branch e os commits.

Após todos os testes passarem, execute:

git switch main

git merge validacao-dados

git push origin main

Mantenha a branch validacao-dados como registro da atividade.

RESULTADOS E ENTREGAS ESPERADOS
Entregue o link do mesmo repositório abertura-ordens-servico, com as validações concluídas, commits das etapas e as branches main e validacao-dados atualizadas no GitHub. Exporte a collection “Abertura de Ordens de Serviço”, com testes de POST e PUT válidos e inválidos para Setor e Equipamento. Documente as respostas e confirme que requisições rejeitadas não cadastraram nem alteraram dados. Na demonstração, explique a função de @NotBlank, @NotNull e @Valid e a diferença entre exigir um setor e verificar se ele existe no banco. Inclua os testes das mensagens personalizadas e explique como o ValidacaoExceptionHandler reúne os campos e suas mensagens. Demonstre uma resposta com vários campos inválidos.

Material de Consumo:	Não se aplica
Ferramentas:	IDE; MySQL Workbench; Postman; Git e GitHub; Spring Boot
EPIs:	Não se aplica
EPCs:	Não se aplica
Equipamentos:	Computador com Java/JDK, MySQL e acesso às dependências do projeto
Instrumentos:	Não se aplica